

Supplementary Materials for Visibility-Aware Cooperative Tracking with Decentralized LiDAR-Based Aerial Swarms

Longji Yin, Yunfan Ren, Fangcheng Zhu, Liuyu Shi, Fanze Kong,
Benxu Tang, Wenyi Liu, Ximin Lyu, and Fu Zhang

I. L_1 - L_2 SSDF UPDATE DERIVATION

This section provides the mathematical derivation showing that updating the spherical signed distance field requires sequential L_1 and L_2 distance transforms, and proves that the SSDF satisfies the properties required by the L_2 transform algorithm in [1].

A. Problem Formulation

Let C denote the spherical distance between $\{\theta, \phi\}$ and $\{\theta', \phi'\}$. The spherical law of cosines gives:

$$\cos C = \cos \theta' \cos \theta + \sin \theta' \sin \theta \cos |\phi - \phi'|. \quad (1)$$

Updating the spherical distance for a given point $\{\theta, \phi\}$ requires finding its closest boundary point $\{\theta', \phi'\}$ in the visible set U . To minimize the angular distance C , we maximize $\cos C$:

$$D(\theta, \phi) = \max_{\theta', \phi'} \{ \cos \theta' \cos \theta + \sin \theta' \sin \theta \cos |\phi - \phi'| + f(\theta', \phi') \}. \quad (2)$$

where $f(\theta', \phi')$ is a barrier function constraining $\{\theta', \phi'\}$ to the visible set U :

$$f(\theta', \phi') = \begin{cases} 0, & \{\theta', \phi'\} \in U, \\ -\infty, & \{\theta', \phi'\} \notin U. \end{cases} \quad (3)$$

This can be decomposed into nested optimizations:

$$D(\theta, \phi) = \max_{\theta'} \{ \cos \theta' \cos \theta + \max_{\phi'} \{ \sin \theta' \sin \theta \cos |\phi - \phi'| + f(\theta', \phi') \} \}. \quad (4)$$

B. Inner Optimization: L_1 Distance Transform

For the inner optimization over ϕ' , given fixed θ, ϕ , and θ' :

$$\phi'_{max} = \arg \max_{\phi': \{\theta', \phi'\} \in U} \{ \sin \theta' \sin \theta \cos |\phi - \phi'| \}. \quad (5)$$

Since $\theta, \theta' \in [0, \pi]$ in spherical coordinates, we have $\sin \theta' \sin \theta \geq 0$. Thus the inner maximization simplifies to:

$$\phi'_{max} = \arg \max_{\phi': \{\theta', \phi'\} \in U} \{ \cos |\phi - \phi'| \}. \quad (6)$$

Since $|\phi - \phi'| \in [0, \pi]$ and $\cos(\cdot)$ is monotonically decreasing on $[0, \pi]$:

$$\phi'_{max} = \arg \min_{\phi': \{\theta', \phi'\} \in U} \{ |\phi - \phi'| \}. \quad (7)$$

This is a standard L_1 -norm problem, solvable via forward-backward scan.

C. Outer Optimization: L_2 Distance Transform

After obtaining $\phi'_{max}(\theta')$ for each latitude θ' , the outer optimization becomes:

$$D(\theta, \phi) = \max_{\theta'} \{ \cos \theta' \cos \theta + \sin \theta' \sin \theta \cos |\phi - \phi'_{max}(\theta')| \}. \quad (8)$$

For a fixed longitude ϕ , each candidate boundary point $\{\theta', \phi'_{max}(\theta')\}$ induces a cost curve:

$$g_{\theta'}(\theta) = \cos \theta' \cos \theta + \sin \theta' \sin \theta \cos |\phi - \phi'_{max}(\theta')|. \quad (9)$$

The outer optimization computes the upper envelope:

$$D(\theta, \phi) = \max_{\theta'} g_{\theta'}(\theta). \quad (10)$$

D. Single Intersection Property for L_2 Transform

The L_2 algorithm in [1], [2] requires the following property: **Single Intersection Property:** For any two distinct latitudes $\theta'_1 < \theta'_2$, there exists exactly one latitude θ_0 such that $g_{\theta'_1}(\theta_0) = g_{\theta'_2}(\theta_0)$, and:

$$\begin{cases} g_{\theta'_1}(\theta) > g_{\theta'_2}(\theta) & \text{when } \theta < \theta_0, \\ g_{\theta'_1}(\theta) < g_{\theta'_2}(\theta) & \text{when } \theta > \theta_0. \end{cases} \quad (11)$$

This property ensures that each cost curve contributes at most one contiguous interval to the upper envelope, enabling linear-time construction.

E. Proof of Single Intersection Property

Consider two distinct latitudes $\theta'_1 < \theta'_2$ with cost curves:

$$g_{\theta'_1}(\theta) = \cos \theta'_1 \cos \theta + \sin \theta'_1 \sin \theta \cos |\phi - \phi'_1|, \quad (12)$$

$$g_{\theta'_2}(\theta) = \cos \theta'_2 \cos \theta + \sin \theta'_2 \sin \theta \cos |\phi - \phi'_2|, \quad (13)$$

where $\phi'_1 = \phi'_{max}(\theta'_1)$ and $\phi'_2 = \phi'_{max}(\theta'_2)$. Define the difference function:

$$\Delta G(\theta) = g_{\theta'_1}(\theta) - g_{\theta'_2}(\theta). \quad (14)$$

Expanding:

$$\begin{aligned} \Delta G(\theta) = & (\cos \theta'_1 - \cos \theta'_2) \cos \theta \\ & + (\sin \theta'_1 \cos |\phi - \phi'_1| - \sin \theta'_2 \cos |\phi - \phi'_2|) \sin \theta. \end{aligned} \quad (15)$$

Let $\alpha = \sin \theta'_1 \cos |\phi - \phi'_1| - \sin \theta'_2 \cos |\phi - \phi'_2|$ and $\beta = \cos \theta'_1 - \cos \theta'_2$. Then:

$$\Delta G(\theta) = \alpha \sin \theta + \beta \cos \theta. \quad (16)$$

Since $\theta'_1 < \theta'_2$ and $\cos(\cdot)$ is monotonically decreasing on $(0, \pi)$, we have $\beta > 0$. Using the harmonic addition theorem:

$$\Delta G(\theta) = \sqrt{\alpha^2 + \beta^2} \cos(\theta - \psi), \quad (17)$$

where $\psi = \arctan(\alpha/\beta) \in (-\pi/2, \pi/2)$ since $\beta > 0$.

At the intersection point θ_0 where $\Delta G(\theta_0) = 0$, we have $\cos(\theta_0 - \psi) = 0$. Given $\theta_0 \in (0, \pi)$ and $\psi \in (-\pi/2, \pi/2)$, this yields $\theta_0 = \psi + \pi/2 \in (0, \pi)$.

Since $\cos(\cdot)$ is positive on $(-\pi/2, \pi/2)$ and negative on $(\pi/2, 3\pi/2)$:

- When $\theta < \theta_0$: $\theta - \psi < \pi/2$, thus $\cos(\theta - \psi) > 0$, hence $\Delta G(\theta) > 0$ and $g_{\theta'_1}(\theta) > g_{\theta'_2}(\theta)$.
- When $\theta > \theta_0$: $\theta - \psi > \pi/2$, thus $\cos(\theta - \psi) < 0$, hence $\Delta G(\theta) < 0$ and $g_{\theta'_1}(\theta) < g_{\theta'_2}(\theta)$.

This establishes the single intersection property, and hence the outer optimization can be solved using the L_2 -distance transform algorithm from [1].

II. COMPLEXITY ANALYSIS OF VISIBILITY MAP AND SSDF CONSTRUCTION

This section provides a complexity analysis of the visibility map construction and SSDF update algorithms described in Sec. IV of the main text.

A. Visibility Map Construction vs. Ray-Tracing Construction

Classical ray-tracing casts rays from the focal point along each discretized direction (θ_i, ϕ_j) and steps through voxels to find the first intersection, yielding a complexity of $O(N_\theta N_\phi \cdot L/\delta)$, where L is the ray length and δ is the step size. Our visibility map update (Alg. 1) instead traverses occupied cells in the occupancy map and records the minimum occluded radius k_{min} for each direction. Leveraging the monotonic property of LOS visibility (all positions beyond k_{min} are directly occluded), the complexity reduces to $O(N_{occ} + N_\theta N_\phi)$, where N_{occ} is the number of occupied cells. This is typically more efficient since $N_{occ} < N_\theta N_\phi \cdot (L/\delta - 1)$ when occupied cells are sparse relative to the total traversed voxels, which is common in most environments.

B. SSDF Construction

The SSDF construction consists of a 2-D spherical distance transform followed by a 3-D incremental update, as described in Secs. IV-B and IV-C of the main text.

The 2-D spherical distance transform (Algs. 2–3) applies two sequential scans over the θ - ϕ grid. Both the latitudinal scan (Alg. 2) and the longitudinal scan (Alg. 3) have linear complexity in the grid size, yielding an overall complexity of $O(N_\theta \cdot N_\phi)$ per radial layer [1], [2].

The 3-D incremental update (Alg. 4) processes layers sequentially from the outermost to the innermost. At each layer, only the newly visible directions are inserted via BFS-based propagation [3]. The complexity of this stage is

$O(N_{change} \log N_{change})$, where N_{change} is the total number of state-changing cells across all layers. Since the monotonic property ensures that visibility changes are incremental between adjacent layers, N_{change} is typically much smaller than the θ - ϕ grid size $N_\theta \cdot N_\phi$.

The overall SSDF construction complexity is $O(N_\theta \cdot N_\phi + N_{change} \log N_{change})$, which is manageable for real-time on-board computation. Runtime measurements are provided in Sec. VIII-E of the main text.

III. ANALYSIS OF TWO-STAGE PLANNING PIPELINE

This section clarifies the separation of responsibilities between the front-end primitive searcher and the back-end trajectory optimizer, and quantifies how the trajectory deforms and how costs change during optimization.

A. Role of Each Planning Stage

The front-end primitive searcher is responsible for topological path guidance, determining which side to pass obstacles and how to navigate around occluded regions based on the evaluated costs. Safe flight corridors are then constructed along the topological path, within which the back-end trajectory is confined. The primitive nodes of the front-end path also serve as the initial values for back-end trajectory optimization. The back-end optimizer then performs fine-grained trajectory refinement based on the same cost formulation, yielding a high-quality and dynamically feasible final trajectory.

B. Trajectory Deformation and Cost Changes

To quantify the deformation and cost changes between planning stages, we conduct tracking tests and record an example of optimization process. Fig. 1 illustrates the front-end path nodes (black dots), the initial trajectory derived from the front-end path (blue), and the optimized final trajectory (red) in one planning cycle of UAV1. The corresponding cost changes from the initial to the optimized trajectory are recorded in Table I, listing several key optimization cost terms.

From the example, we observe that the total cost is substantially reduced after optimization. The angular velocity cost is also effectively regulated, and the control effort is reduced, resulting in a smoother trajectory. This is expected as the discrete front-end may produce abrupt acceleration changes that rely on the back-end to smooth out. The FOV cost is also substantially optimized in the back-end (from 4479.9 to 0.0), since the discrete front-end cannot effectively handle such high-order constraints. Meanwhile, the front-end path already adequately addresses certain constraints, such as swarm collision avoidance and obstacle occlusion (SSDF remains 0.0), and the gradient-based back-end optimization ensures that these satisfied cost terms do not deteriorate. The consistency between the front-end and back-end is theoretically guaranteed by the unified cost formulation shared across both stages, with the back-end refinements primarily compensating for the front-end's discrete nature and its limited ability to handle high-order constraints.

TABLE I
KEY COST CHANGES IN TRAJECTORY OPTIMIZATION

Metrics	Total	Control Effort	Safe Corridor	Angular Velocity	Swarm Collision	Target Distance	SSDF	FOV
Initial Value	13717.1	5350.3	0.0	4874.2	430.4	1958.7	0.0	4479.9
Final Value	280.4	1401.9	0.0	0.0	0.0	0.0	0.0	0.0

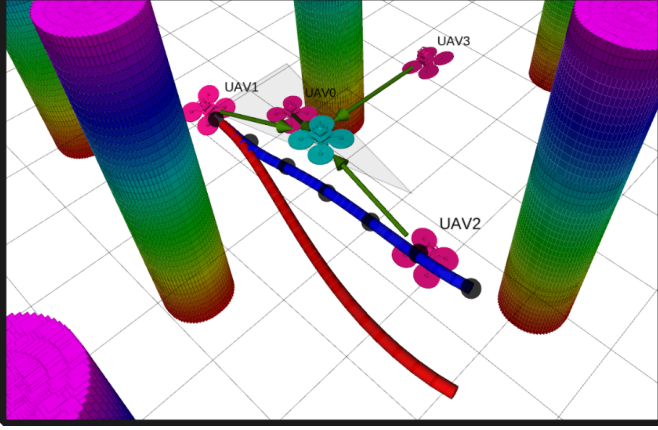


Fig. 1. Trajectory deformation from front-end initialization (blue) to back-end optimized result (red). Black dots indicate the front-end path nodes.

TABLE II
TRACKING RESULTS WITH AVIA LiDARS

Metric Scene	Avg. Swarm Visibility (ϑ_{avg})	Full Vis. Ratio (γ_{vis})	Avg. Horiz. Align. Angle
Density 1/32	5.000	100.00%	5.321°
Density 1/25	4.995	99.47%	5.638°
Density 1/18	4.980	98.05%	6.775°
Density 1/14	4.958	95.88%	7.321°
Density 1/9	4.878	89.54%	7.886°

IV. EVALUATION WITH LIMITED-FOV LiDARS

The main experiments use horizontally-omnidirectional LiDARS. To evaluate the applicability of our method to limited-FOV sensors, we conduct additional experiments with Livox Avia LiDARS, which have a conic (non-horizontally-omnidirectional) field of view. Specifically, a swarm of five UAVs equipped with Avia LiDARS is tested using the ablation study settings from Sec. VIII-C.

In addition to the average swarm visibility (ϑ_{avg}) and full visibility ratio (γ_{vis}), we record the horizontal FOV alignment performance, measured as the average angle between the horizontal FOV center axis and the target direction. This angle equals 0 when perfectly aligned, and the target remains horizontally sensible when this angle is below 35° (half of Avia's horizontal FOV). Table II summarizes the tracking results across different obstacle densities. The average horizontal alignment angle remains below 8 in all scenarios, well within the sensible range. Fig. 2 shows a snapshot of the tracking process. These results demonstrate that our FOV-aware planner effectively orients the drones toward the target, maintaining visibility within the limited horizontal FOV throughout the tracking task.

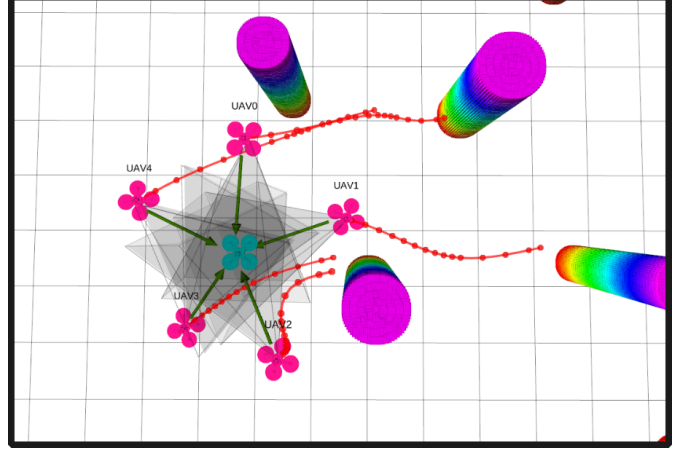


Fig. 2. A snapshot of tracking with conic-FOV Avia LiDARS.

V. DISCUSSION ON MULTI-MODAL TARGET ESTIMATION

The target state estimation in our system uses a constant-velocity error-state Kalman filter (ESKF), as described in Sec. III-B of the main text. Since the Kalman filter (KF) maintains a unimodal distribution, it cannot represent multiple hypotheses on the target location. During occlusion, the KF performs prediction only, with the state estimate μ extrapolated linearly and the covariance Σ growing over time. When measurements resume, the enlarged Σ yields a high Kalman gain that enables substantial correction. Brief occlusions allow smooth recovery, but prolonged occlusion may require several update cycles to re-lock onto the target, as the unimodal estimate must be corrected from a potentially distant position.

In practice, our system actively prevents the continuous measurement loss through multiple mechanisms: (1) SSDF-based visibility and FOV costs enable proactive occlusion avoidance; (2) decentralized multi-drone measurement fusion provides measurement redundancy; (3) the swarm distribution cost coordinates multi-angle coverage, reducing the probability of simultaneous all-tracker occlusion; and (4) high-frequency replanning at 15 Hz enables rapid response before target loss. As shown in Table I of the main text, the worst-case visibility ϑ_{wst} remains well above zero throughout all experiments, and simultaneous target loss of all trackers never occurred. Consequently, the unimodal KF serves as a simple and efficient solution sufficient for the current system.

Nevertheless, to demonstrate the potential benefit of multi-modal estimation under prolonged occlusion, we implement a particle filter (PF) as an alternative. Unlike the KF, the PF represents the target distribution using a set of weighted particles, naturally supporting multi-modal distributions. During occlu-

sion, particles diffuse according to the motion model, covering various possible target locations. When measurements resume, particles near the true position receive high likelihood weights and are retained through resampling, enabling rapid recovery.

We demonstrate the PF in a simulation where a single tracker follows a target drone in an environment with cylinder obstacles using LiDAR point cloud measurements. The tracker is constrained to maintain approximately 2.5 m lateral distance from the target without any visibility-awareness, deliberately allowing occlusions to occur. Fig. 3 shows an example segment of the tracking process. During occlusion, the target shifts laterally while unobserved. The particles diffuse to cover multiple possible positions, representing multiple hypotheses. When the tracker passes the obstacle and re-observes the target, the particles rapidly converge to the measured position through resampling. This fully anonymous video¹ shows the simulated tracking process with the particle filter.

Table III quantitatively compares the KF and PF performance in the tracking using three metrics: average position error, maximum error, and average recovery time (defined as the duration from measurement resumption until the estimation error drops below 0.2 m). The PF achieves notably faster recovery than the KF, validating the advantage of multi-modal estimation under prolonged occlusion. However, given that our system actively ensures continuous target observation, complete visibility loss rarely occurs in practice, making the efficient KF sufficient for deployment.

TABLE III
COMPARISON OF KF AND PF ESTIMATION PERFORMANCE

Method	Avg. Error (m)	Max. Error (m)	Recovery Time (ms)
PF	0.145	0.878	87
KF	0.154	1.292	374

REFERENCES

- [1] P. F. Felzenszwalb and D. P. Huttenlocher, “Distance transforms of sampled functions,” *Theory of computing*, vol. 8, no. 1, pp. 415–428, 2012.
- [2] R. Wang, M. Pan, X. Han, W. Chen, and H. Bao, “Parallel and adaptive visibility sampling for rendering dynamic scenes with spatially varying reflectance,” *Computers & graphics*, vol. 38, pp. 374–381, 2014.
- [3] L. Han, F. Gao, B. Zhou, and S. Shen, “Fiesta: Fast incremental Euclidean distance fields for online motion planning of aerial robots,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 4423–4430.

¹<https://youtu.be/hkoJSPNpc1U>

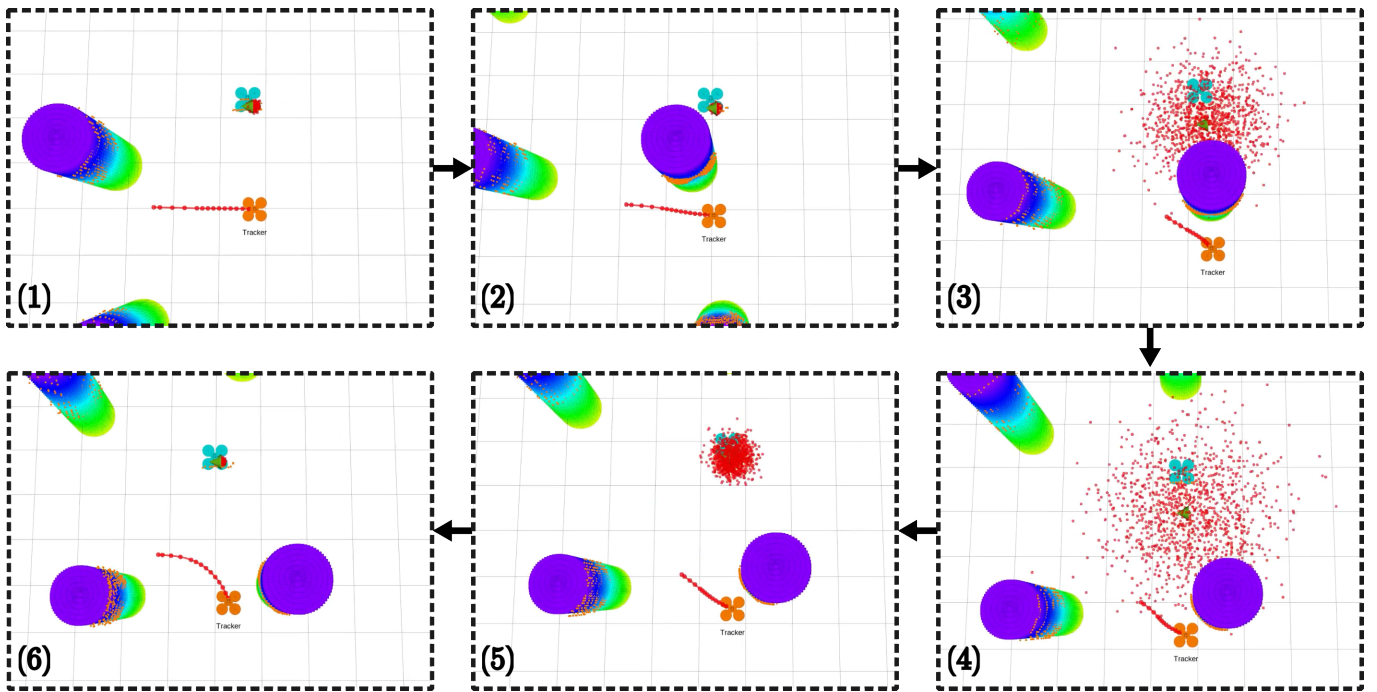


Fig. 3. Example sequence of the tracking process with particle filter under occlusion. The orange and blue drones represent the tracker and target respectively. Red points visualize the particles, and the green arrow indicates the estimated target position. Orange points on the obstacles and target drone represent the LiDAR point cloud from the tracker.